

DEEPL - TP 3

Recurrent Networks

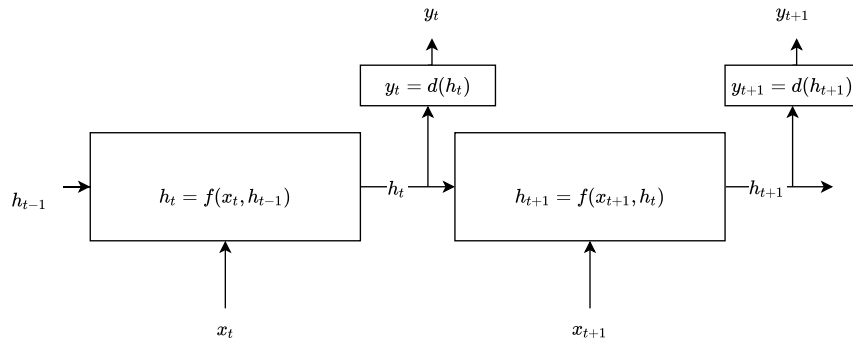
Nicolas Baskiotis - Stéphane Rivaud - Benjamin Piwowarski - Laure Soulier

2025-2026

1 Introduction (brief, see lecture)

Recurrent neural networks (RNN) are a type of architecture favored for processing sequences. Generically, a recurrent network uses a hidden state (or *memory*) to accumulate information from previous iterations. When computing at a time t in the sequence, the network uses both the input at time t and the hidden state to infer a new hidden state. The hidden state can be decoded to predict an output value corresponding to time t but can also be used to continue inference at the next time step.

Formally, a recurrent network f computes the next state through an encoding step : $\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1})$, with $\mathbf{x}_t$ the input at time t , $\mathbf{h}_{t-1}$ the hidden state from the previous time step, $\mathbf{h}_t$ the hidden state produced by the network from $\mathbf{x}_t$ and $\mathbf{h}_{t-1}$. A decoder d allows inferring the output of interest $\mathbf{y}_t$ from the hidden state $\mathbf{h}_t$.



A simple example of a network is $\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}) = \tanh(\mathbf{W}_i \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b}_h)$ which performs a linear transformation on the inputs (*input*) defined by the weights $\mathbf{W}_i$, one on the hidden state (*hidden*) defined by the weights $\mathbf{W}_h$, combines the two then applies a non-linearity (hyperbolic tangent, sigmoid or ReLU for example). The hidden state can also be decoded with a linear network combined with a non-linearity : $\mathbf{y}_t = d(h_t) = \sigma(\mathbf{W}_d \mathbf{h}_t + \mathbf{b}_d)$.

For example, let $\mathbf{x} = \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$ be an input sequence (possibly multivariate : $\mathbf{x}_i \in \mathbb{R}^d$) and $\mathbf{h}_0$ an initial hidden state. To compute the decoding of this sequence, the steps are as

follows :

$$\begin{array}{llll} \mathbf{h}_1 = f(\mathbf{x}_1, \mathbf{h}_0) & \mathbf{h}_2 = f(\mathbf{x}_2, \mathbf{h}_1) & \dots & \mathbf{h}_T = f(\mathbf{x}_T, \mathbf{h}_{T-1}) \\ \mathbf{y}_1 = d(\mathbf{h}_1) & \mathbf{y}_2 = d(\mathbf{h}_2) & \dots & \mathbf{y}_T = d(\mathbf{h}_T) \end{array}$$

Other multiple possibilities exist for computing the hidden state : for example, having a common network instead of two separate linear networks for inputs and state. In this case, a concatenation of inputs and hidden state is performed and a single weight matrix is used on this concatenated vector. We will see more complex architectures in the following practical sessions.

From this general principle, RNNs are very flexible and multiple applications exist :

- the hidden state can be decoded only at the end of a sequence to produce a single output. This type of network, called *many-to-one* (figure 1), allows for example to classify a sequence : the output decoding then indicates the class of the sequence ;

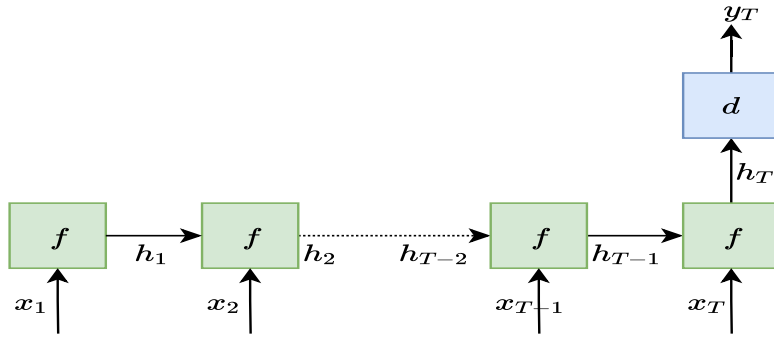


FIGURE 1 – Example of many-to-one network

- the hidden state can be decoded at each time step (type *many-to-many*, figure 2), for forecasting (prediction of temperatures, traffic jams) or text generation ;

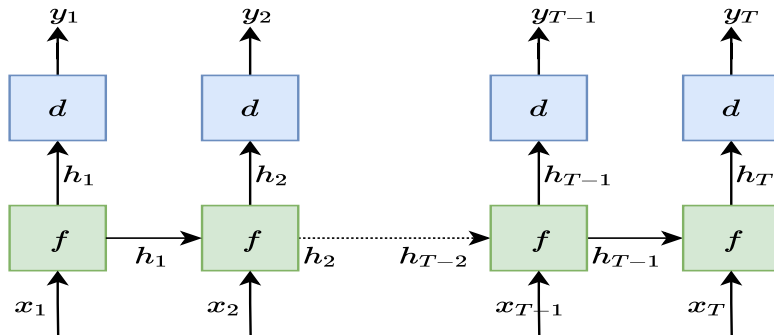


FIGURE 2 – Example of many-to-many network

- a variant (figure 3) consists of reading an entire sequence then producing a sequence from the final hidden state (particularly for translation) ; in this case, when generating the new sequence, the decoded element from the previous step y_t must be introduced (since there is no corresponding input x_t).

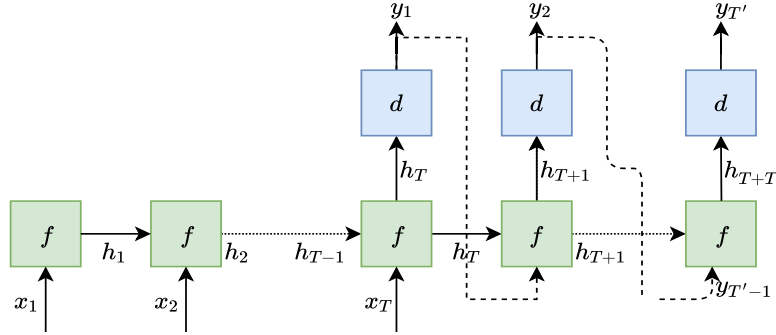


FIGURE 3 – Example of many-to-many network, variant

- there can also be only one input from which a sequence is produced (type *one-to-many*, figure 4).

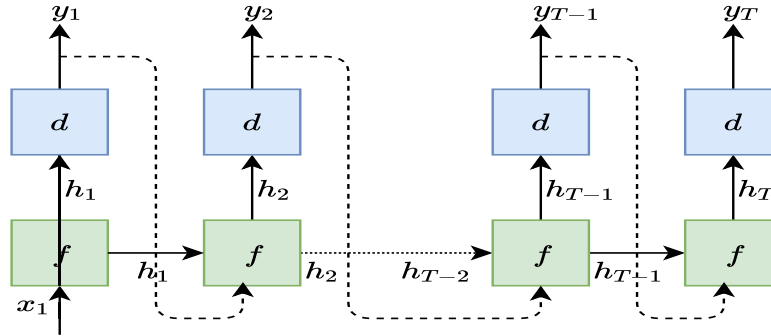


FIGURE 4 – Example of one-to-many network

In the rest of this practical session, we will successively study the problem of sequence classification (architecture of figure 4), forecasting (architecture 2) and finally generation (architecture 2 also). The other architectures will be studied in the following practical sessions.

Important

In all that follows, we will consider that our models can take as input a batch of sequences. The dimension of the input will always be $length \times batch \times dim$ with $length$ the length of sequences, $batch$ the batch size and dim the dimension of an element of the sequence. We will limit ourselves to sequences of fixed sizes per batch (they can be of different sizes for different batches) to facilitate batch construction.

2 Implementing an RNN

Important

The architectures presented above all use the same RNN model. The difference is in the supervision, the use of hidden states and the decoding. In what follows, you are therefore asked to implement **a single RNN class** and depending on the targeted application, your training loop and the decoder used will change.

For a network whose hidden state is of dimension `latent`, the input of dimension `dim` and the desired output of dimension `output`, the class must include :

- a function `one_step(x,h)` that allows processing one time step : it takes as input a batch `x` at a time t of sequences of size $batch \times dim$ and the batch of hidden states of size $batch \times latent$, and returns the following hidden states of size $batch \times latent$.
- the method `forward(x,h)` which must process the entire batch of sequences passed as parameter. It successively calls the `one_step` method on all elements of the sequences. The size of `x` is $length \times batch \times dim$ and `h` of size $batch \times latent$. It must return the sequence of computed hidden states of size $length \times batch \times latent$.
- a method `decode(h)` that allows decoding the batch of hidden states, with `h` of size $batch \times latent$ and returns a tensor of size $batch \times output$.

Question 1

Code an RNN class that implements a simple recurrent network as described in the introduction.

3 Sequence Classification

Download the Hangzhou metro dataset. It describes the incoming and outgoing flow of 80 stations in the city aggregated by quarter hour between 5 : 30 AM and 11 : 30 PM each day. Two tensors are in the archive, one for training and the other for testing. They are of size $D \times T \times S \times 2$ with D the number of days, $T = 73$ the successive quarter-hour intervals between 5 : 30 AM and 11 : 30 PM, $S = 80$ the number of stations and the incoming and outgoing flows for the last dimension.

The objective of this section is to build a model that from a sequence of a certain length infers the station to which this sequence belongs. Typically, this type of problem is impossible to handle with a classical network : even considering sequences of fixed length, the dimensions are not homogeneous across sequences (a dimension does not always represent the same quarter hour).

Question 2

Test your implementation on the dataset, initially selecting a subset of about ten stations. You can initially use only the incoming flow, then the incoming and outgoing flow in a second phase.

For decoding, as the objective is to perform multi-class classification, it is usual to use a linear layer, followed by a softmax coupled with a cross-entropy cost.

You will consider for each batch sequences of the same length (you can vary the sequence length between batches). Furthermore, you can use a zero tensor for the initial state. Observe the cost in training and testing, as well as the error rate (0-1 cost). Vary the sequence length for testing.

In this context of *many-to-one* network, supervision is done only at the end of the sequence (and not at each instant of the sequence), when the class is decoded. The associated error is backpropagated through all the traversed states.

4 Multivariate Sequence Model

The objective of this part is to do time series prediction : from a flow sequence of length t for all stations in the dataset, we want to predict the flow at $t + 1, t + 2, \dots$. You will train a common RNN for all stations that takes a series in $\mathbb{R}^{n \times 2}$ and predicts a series in $\mathbb{R}^{n \times 2}$.

Question 3

What should be changed in the previous model? What cost is in this case more suitable than cross-entropy?

Conduct experiments by varying the prediction horizon (at $t + 2$, etc.) and the length of input sequences. You can as before first consider only the incoming flow, then the incoming and outgoing flow.

In this context of *many-to-many* network, supervision can be done at each step of the sequence without waiting for the end of the sequence. Backpropagation is done only once the entire sequence has been seen, but at a time t , the gradient takes into account the error at that moment (based on the decoding supervision) but also the error from subsequent time steps which is accumulated.

5 Sequence Generation

Principle Sequence generation consists of producing a sequence of discrete symbols from an input sequence. The objective is therefore to decode from the hidden state a multinomial probability distribution over the symbols to generate. The output dimension of the RNN must therefore be equal to the number of symbols considered. Furthermore, a softmax must be used to obtain a distribution from the hidden state decoding. The cross-entropy cost is suitable for learning this distribution.

Embedding When the input sequence is also discrete, the input space must first be projected into a continuous space - what is called an *embedding*. Suppose we have n symbols to encode. A first step is to do an encoding called *one-hot* in $\mathbb{R}^n$: each symbol is assigned an index (thus obtaining a symbol dictionary) and is represented in a continuous space by a vector of dimension n zero everywhere except in the dimension of its index set to 1. The second step consists of learning a representation in a second space of dimension n' (with $n' < n$ in general - for the case of characters, there will not be a big difference) through a linear projection. This learning can be done independently of the generation task or more classically at the same time as learning the general network (which you will do during this practical session). This representation can then be used as input to the network.

Generation Once the network is learned, generation is done (either from a sequence beginning, or from an empty initial state) by choosing the most probable symbol in the decoded multinomial distribution at each time step. This symbol is then considered as input at the next time step and generation continues iteratively. Another possibility is to sample from the multinomial to obtain several samples.

In this practical session, we will limit ourselves to generating sequences of **fixed size** that we will determine in advance.

Question 4

The second provided dataset is a set of Trump's pre-election speeches. The goal is to learn an RNN that allows generating a Trump-style speech.

The code for text cleaning (removing accented and unusual characters) and transforming a character string into a list of indices is provided in the skeleton.